Parallel MMUL & LIKWID Performance Counters Assignment #4, CSC 746, Fall 2026

Jacob Cordano
Computer Science Department
San Francisco State University
San Francisco, CA, USA
Email: jcordano1@sfsu.edu

***Abstract*—Matrix multiplication is computationally expensive and sensitive to parallel scalability and memory locality. We implemented two matrix multiplication algorithms with OpenMP, one of which is a basic row parallel triple nested loop and another with cache blocking and thread local tile copies. Both algorithms were evaluated on the NERSC Perlmutter with block sizes of 128, 512, and 2048 using threads between 1 and 64, and measured with the Like I Know What I Knew What I'm Doing (LIKWID) API for runtime and cache utilization. At N = 2048, the basic and blocked implementations achieved 51x and 40x speedup at 64 threads, while the blocked implementation ran at 6x faster. The blocked algorithm also had significantly reduced L2 and L3 accesses relative to the basic implementation.**

I. Introduction

Matrix multiplication computation with performance that is effective with both parallel execution and has efficient use of memory hierarchy is crucial. This study examines hoe OpenMP parallelism and cache blocking affect the runtime, scalability, and cache behavior of matrix multiplication.

We implemented a triple nested loop that parallelizes independent output rows and a blocked implementation that operates on thread local matrix tiles. Both implementations compute C = C + AB and use LIKWID instrumentation to isolate the multiplication region. These were evaluated on NERSC Perlmutter using matrix sizes of 128, 512, and 2048, with thread counts of between 1 and 64, differentiating block sizes, and referenced CBLAS for cache and instruction count comparisons.

These implementations scaled most effectively for the largest problem size, where the basic algorithm reached a 51x speedup and the blocked implementation reaching 40x at a thread count of 64. LIKWID showed that blocking with a size of 16 reduced the L2 and L3 cache accesses significantly compared to the basic algorithm.

II. Implementation

Here, we describe the implementation of the vector matrix multiplication algorithms (VMM) that we have developed and evaluated in this study. We first implement a basic matrix multiplication algorithm that has been parallelized using OpenMP. We then implement a blocked matrix multiplication algorithm that has also been parallelized using OpenMP.

A. DGEMM Basic Open MP Implementation

The basic DGEMM implementation performs square matrix–matrix multiplication, computing $C = C + AB$. The matrices are stored in row-major order. OpenMP parallelizes the outer loop over the rows of A, allowing different threads to update distinct rows of C concurrently. For each output element C_{ij}, the innermost loop computes the dot product of row i of A and column j of B, adding the result to the existing value of C_{ij}. Because each OpenMP iteration updates a distinct row of C, no synchronization is required for the matrix updates. The LIKWID markers enclose the parallel computation for performance measurement.

```
1 void square_dgemm(int n, double *A, double *B,
      double *C){
2     #pragma omp parallel
3     {
4         LIKWID_MARKER_START(
    MY_MARKER_REGION_NAME);

6 #pragma omp for
7         for (int i = 0; i < n; i++)
8         {
9             for (int j = 0; j < n; j++)
10            {
11                double sum = C[i * n + j];
12                for (int k = 0; k < n; k++)
13                {
14                    sum += A[i * n + k] * B[k
    * n + j];
15                }
```

```
16                  C[i * n + j] = sum;
17              }
18          }

20          LIKWID_MARKER_STOP(
        MY_MARKER_REGION_NAME);
21      }
22  }
```

Listing 1. Basic DGEMV OMP implementation

B. DGEMM Blocked Open MP Implementation

The blocked DGEMM computes $C = C + AB$ for $n \times n$ matrices A, B, and C, all stored in row-major order. The matrices are divided into square tiles with dimensions `block_size` by `block_size`. Each tile of C is copied into a thread-local buffer and updated using the corresponding tiles of A and B, which are also copied into thread-local buffers. This copy optimization improves spatial and temporal locality by allowing the block multiplication to operate on contiguous data that can be reused from the cache. OpenMP parallelizes the outer loop over block rows, allowing threads to update distinct regions of C concurrently without requiring synchronization. After all contributing block products have been accumulated, the updated tile is copied back into C. The LIKWID markers enclose the parallel computation for performance measurement.

```
1 PROCEDURE Blocked_DGEMM(n, size, A, B, C)
2     blocks = n / size

4     PARALLEL FOR each block (i, j)
5         Copy C(i,j) to local C_block

7         FOR k = 0 TO blocks - 1
8             Copy A(i,k) to local A_block
9             Copy B(k,j) to local B_block
10            C_block = C_block + A_block *
      B_block
11        END FOR

13        Copy C_block back to C(i,j)
14    END PARALLEL FOR
15 END PROCEDURE
```

Listing 2. Blocked DGEMV OMP pseudocode implementation

III. Results

A. Computational platform and Software Environment

This evaluation was performed on the NERSC Perlmutter supercomputer using two AMD EPYC 7763 processors with a base clock speed of 2.45 GHz. The system was configured with 512 GB of DDR4 memory per node, a 32 KB L1 cache, a 512 KB L2 cache, and a 32 MB L3 cache, providing ample resources for large matrix multiplications. The operating system is SUSE Linux Enterprise Server 15 SP6 with kernel version `6.4.0-150600.23.125_15.0.29-cray_shasta_c`. GCC 14.3.0 and CMake 3.28.3 were used for compilation and build configuration, respectively. [3]

B. Methodology

While conducting this evaluation, we focused on measuring the performance of our matrix multiplication implementations under various configurations. We utilized OpenMP for parallelization and compared the performance of our Basic Matrix Multiplication, Blocked Matrix Multiplication, and the CBLAS library. The evaluation was conducted across different problem sizes, specifically 128, 512, and 2048, to assess scalability and efficiency. We collected data on execution time, speedup, and cache access patterns to provide a comprehensive understanding of the performance characteristics of each implementation.

All metrics were collected using the job files which leveraged the LIKWID performance monitoring tool, allowing us to capture detailed information about speedup using the Runtime (RDTSC) values as well as cache access patterns using the L2 and L3 cache access counters. The collected data was then analyzed to identify trends, bottlenecks, and areas for optimization.

C. Scaling Study for Basic MM with OpenMP parallelization

The scaling study for basic matrix multiplication with OpenMP parallelization was conducted to evaluate the performance improvements achieved through parallel execution. The study involved running the Basic MM implementation across different problem sizes and measuring the speedup achieved by utilizing multiple threads.

To collect this data, we executed the included job-basic-omp file, which was designed to run the Basic MM implementation with OpenMP parallelization. The job file was configured to utilize multiple threads, allowing us to observe the performance gains achieved through parallel execution.

The problem sizes tested were 128, 512, and 2048, and the speedup was calculated by comparing the execution time of the parallel implementation against the serial version. The results of this scaling study are presented in Figure 1, which illustrates the speedup achieved for each problem size.

Speedup was calculated using the formula:

$$\text{Speedup} = \frac{\text{Execution Time (Serial)}}{\text{Execution Time (Parallel)}} \tag{1}$$

The execution time for the parallel implementation was measured using the maximum runtime recorded across all threads, while the serial execution time was obtained from the single-threaded execution of the Basic MM implementation. The reason for using the maximum runtime across threads is to account for the fact that the overall execution time is determined by the slowest thread, which can be influenced by factors such as load imbalance and thread synchronization overhead.

Figure 1 shows the speedup by problem size for the basic matrix multiplication implementation parallelized with OpenMP.

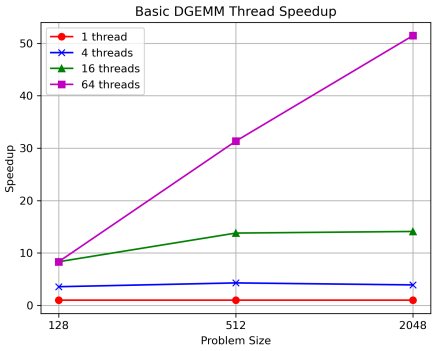


Fig. 1. Speedup of Basic Matrix Multiplication with OpenMP parallelization

We observed that the speedup achieved by the Basic implementation was significant as concurrency increased, with the highest speedup observed for the largest problem size of 2048 and a concurrency of 64 threads. We also observed that speedup was not linear as well and see that problem size seems to have the most significant increases on the 64-thread speedup. This is likely due to the greater ability to utilize the available threads for larger problem sizes.

D. Evaluation of BMMCO with OpenMP parallelization

For the blocked matrix multiplication implementation with OpenMP parallelization, we conducted a similar scaling study to evaluate its performance improvements. The study involved running the blocked implementation across different problem sizes and measuring the speedup achieved by utilizing multiple threads.

To collect this data, we executed the included job-blocked-omp file, which was designed to run the blocked implementation with OpenMP parallelization. The job file was configured to utilize multiple threads, allowing us to observe the performance gains achieved through parallel execution.

To calculate speedup, we used the same formula as in the previous section, comparing the execution time of the parallel implementation against the serial version. The execution time for the parallel implementation was measured using the maximum runtime recorded across all threads, while the serial execution time was obtained from the single-threaded execution of the blocked implementation.

Figure 2 shows the speedup by problem size for the blocked implementation parallelized with OpenMP.

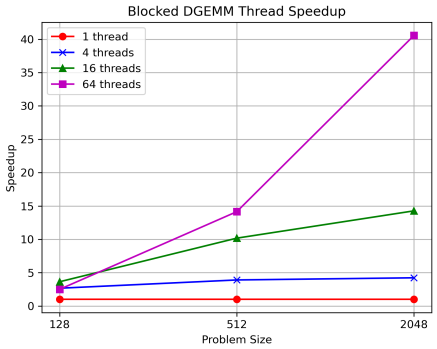


Fig. 2. Speedup of Blocked Matrix Multiplication with OpenMP parallelization

In this observation, we found that speedup was less significant for the blocked implementation compared to the basic implementation. The highest speedup was observed for the largest problem size of 2048 and a concurrency of 64 threads, but the overall speedup was lower than that of the basic implementation. Additionally, we observed that the different problem sizes had generated elevated speedup for the threads less than 64 when compared to the basic implementation.

The lower overall speedup for the blocked implementation can be attributed to the increased complexity of the algorithm compared to the basic implementation, which may introduce additional computational overhead. [1]

E. Comparison of CBLAS, BasicMM-omp, and BMMCO-omp

To compare the L2 and L3 cache performance of the CBLAS library, Basic Matrix Multiplication, and Blocked Matrix Multiplication with OpenMP parallelization, we collected data on the number of cache accesses for each implementation across different problem sizes. The cache access counts were normalized by dividing the counts for Basic and Blocked implementations by the corresponding counts for CBLAS.

Below in Table I, we present the normalized L2 cache access counts for each implementation across the problem sizes of 128, 512, and 2048.

TABLE I
NORMALIZED L2 CACHE ACCESSES

Problem size	CBLAS	Basic	Blocked B4	Blocked B16
128	1.000	44.987	4.574	1.335
512	1.000	45.813	4.449	1.977
2048	1.000	120.906	7.807	2.013

When examining the normalized L2 cache access counts, we observed that the Basic implementation had significantly higher L2 cache accesses compared to CBLAS, indicating that it may not be as efficient in utilizing the cache. The Blocked implementations, particularly with a block size of 16, showed improved cache performance, with lower normalized L2 cache accesses compared to the Basic implementation. The lower L2 cache accesses for the Blocked implementations suggest that they are more effective in leveraging the cache hierarchy, which will lead to faster runtimes.

Furthermore, we also collected data on the normalized L3 cache access counts for each implementation across the same problem sizes. The results are presented in Table II.

TABLE II
NORMALIZED L3 CACHE ACCESSES

Problem size	CBLAS	Basic	Blocked B4	Blocked B16
128	1.000	0.044	0.035	0.053
512	1.000	273.351	13.936	3.043
2048	1.000	604.116	20.994	4.508

The normalized L3 cache access counts reveal that the Basic implementation had significantly higher L3 cache accesses compared to CBLAS in the larger problem sizes, indicating that the algorithm is not as efficient in utilizing the cache hierarchy. Additionally, the blocked implementations demonstrated improved cache performance, with lower normalized L3 cache accesses compared to the Basic implementation, particularly in 512 and 2048 problem sizes. Last, when looking at the problem size of 128, all implementations had similar L3 cache access counts.

Again, the lower L3 cache accesses for the Blocked implementations suggest that they are more effective in leveraging the cache hierarchy, which will lead to faster runtimes. [1] [2]

F. Instruction Count Analysis

Table III compares the serial instruction counts reported by the `RETIRED_INSTRUCTIONS` counter. Each value is normalized by the corresponding CBLAS instruction count for the same problem size.

TABLE III
NORMALIZED RETIRED INSTRUCTION COUNTS

Problem size	CBLAS	Basic	Blocked B4	Blocked B16
128	1.000	9.511	25.853	11.860
512	1.000	10.561	28.960	13.326
2048	1.000	10.641	29.241	13.466

When evaluating the instruction counts, we can see that the Basic implementation had the least number of instructions retired, which is expected as it is the simplest implementation. The Blocked implementations had higher instruction counts, with the Blocked B4 implementation having the highest counts. The high instruction counts for the blocked implementations can be attributed to the additional computations when performing titled computations.[2] This is especially relevant for the block size of 4, which has the highest instruction counts due to the increased number of iterations and computations required for smaller blocks. The Blocked B16 implementation had lower instruction counts compared to Blocked B4, indicating that larger block sizes can help reduce the number of instructions retired.

G. Overall findings and discussion

- When looking at both speedup charts, we can see that the Basic implementation had the highest speedup for all problem sizes, with the highest speedup observed for the largest problem size of 2048 and a concurrency of 64 threads. The Blocked implementations had lower speedup compared to the Basic implementation, with the Blocked B16 implementation having higher speedup than Blocked B4. The charts are an indication that by limiting complexity of the algorithm and focusing on parallelization, we can achieve better speedup. With those optimizations it also appears that increases in problem size is also achievable without hindering performance.
- As far as caching is concerned with the L2 and L3 access, we can see consistent use of the memory system hierarchy. When we say we make use of the memory hierarchy, we mean maximizing data reuse in the L1 and L2 caches. The basic implementation

fails at cache reuse and this can be evaluated by looking at the high access values for both the L2 and L3 caches. This can similarly be said for the Blocked B16 implementation. On the other end of the spectrum we can see that the Blocked B16 cache utilization is much lower across both L2 and L3 caches.[2]

IV. AI Usage

In order to complete this assignment AI was used in the following ways:

- To configure the included Dockerfile so that I may easily run the homework file without installing various packages locally to my machine.
- After running the benchmarks, I would ask an LLM to combine data into one CSV for plotting.
- For LaTeX debugging, formatting, and learning how to use citations.
- To create my instruction count tables
- To create my cache access tables
- To better understand the lecture materials provided for the course.
- To extract data from the terminal output log files into dedicated csv files.
- To convert my Blocked implementation into pseudocode.

References

[1] Bethel, E. Wes. Csc 746 f26 lecture 13 – cp3 discussion, hw performance counters.pdf, Oct 2026.

[2] Bethel, E. Wes. Csc 746 f26 lecture 14 – cp4, hw perf counters ctd.pdf, Oct 2026.

[3] Nersc. Perlmutter architecture.